

Dispense di Machine Learning

Marco Galletti

A.A. 2024/2025

Sommario

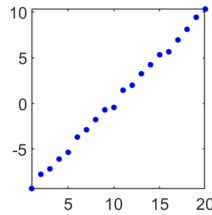
Questa dispensa contiene un riassunto dei principali concetti trattati nel corso di Machine Learning con il professore Emanuele Rodolà del corso di laurea di "Scienze matematiche per l'intelligenza artificiale" presso l'Università Roma "La Sapienza".

Indice

1	Introduzione	1
2	Ripasso algebra lineare	3
3	Regressioni a dipendenza lineare	5
3.1	Regressione lineare	5
3.2	Modelli di regressione polinomiale (regressione lineare con dati polinomiali)	9
3.3	Regressione logistica	10
3.4	Sistema sottodeterminato - regolarizzazione	12
4	Discesa del gradiente	13
4.1	Stochastic Gradient Descent	15
5	Multi-Layer Perceptron & Backpropagation	15
5.1	Grafici computazionali	17
6	Principal Component Analysis	18
6.1	Principal Components	18
6.2	Autovalori e autovettori	20
6.3	Singular Value Decomposition	21
6.4	Principal Component Analysis	22
7	Dimensionality reduction	23
7.1	Spazi metrici	23
7.2	Multidimensional scaling	24
7.3	Stochastic Neighbor Embedding	24
7.4	t-SNE	25
8	Metodi Ensemble	25

1 Introduzione

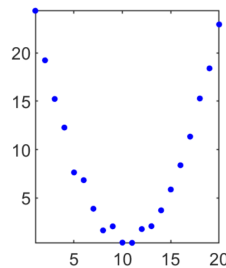
Concetto cardine del machine learning sono i **dati**. Un modello di machine learning è una **funzione**. Ad esempio la per il seguente grafico:



si può supporre un modello lineare, cioè una retta, che meglio approssima i dati.

$$y = ax + b$$

Mentre in quest'altro caso



si osserva che i dati possono essere approssimati meglio da un polinomio di secondo grado.

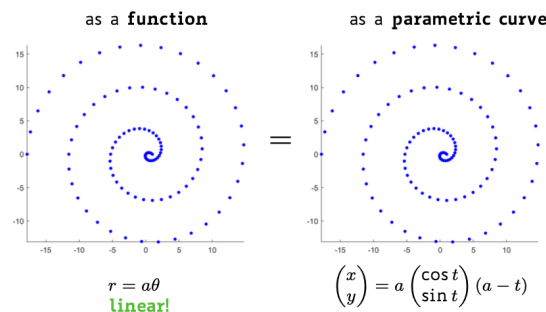
$$y = ax^2 + bx + c$$

Osserviamo che in machine learning noi abbiamo a disposizione dei dati (coppia di valori (x, y)), e l'incognita quindi è la funzione f che meglio approssima i dati. Nel caso specifico, la funzione f è una retta, e quindi è univocamente determinata dai parametri a e b .

Possiamo aggiungere **prior knowledge** al modello, cioè informazioni che possiamo avere sul problema che stiamo studiando, così da ridurre i gradi di libertà del modello, e quindi i parametri incogniti (detti pesi).

Definizione 1.1. I dati che usiamo per addestrare il modello sono detti **training data**. La valutazione del modello avviene su un altro set di dati, detto **test data**.

L'assunzione che prelude al machine learning è che i dati hanno una struttura sottesa da cui il modello deve imparare a cogliere le relazioni tra le variabili. Posso rappresentare i dati oltre che come funzioni anche come curve di parametri. Avviene un trade-off tra il numero dei parametri e la complessità del modello.



Se si hanno pochi elementi in uno spazio si dice **campionamento sparso**. Si congettura il fatto che a un maggior numero di dati corrisponda un migliore outcome di performance.

Definizione 1.2 (Curse of dimensionality). Il **curse of dimensionality** si riferisce alle difficoltà nell'analisi dei dati quando il numero di variabili aumenta. Lo spazio cresce esponenzialmente, rendendo i dati più sparsi, riducendo l'efficacia delle metriche di distanza. Per contrastarla,

si usano tecniche di riduzione della dimensionalità, che comprimono le informazioni mantenendo la loro essenza.

2 Ripasso algebra lineare

Algebra lineare è lo studio di mappe lineari in dimensioni **finite** di spazi vettoriali.

Definizione 2.1 (Spazio vettoriale). Uno spazio vettoriale V è un insieme di elementi detti vettori, dotato di due operazioni:

Commutatività $u + v = v + u \quad \forall u, v \in V$. Inoltre $u + v \in V$.

Associatività $u + (v + w) = (u + v) + w \quad \forall u, v, w \in V$. E $(ab)v = a(bv) \quad \forall u, v, w \in V$ e $\forall a, b \in \mathbb{R}$. Inoltre $av \in V$.

Elemento neutro Esiste un elemento $0 \in V$ tale che $v + 0 = v \quad \forall v \in V$.

Inverso additivo Per ogni $v \in V$ esiste un elemento $w \in V$ tale che $v + w = 0$.

Elemento neutro moltiplicativo Esiste un elemento $1 \in V$ tale che $1v = v \quad \forall v \in V$.

Distributività $a(u + v) = au + av$ e $(a + b)v = av + bv$ per ogni $a, b \in \mathbb{R}$ e $u, v \in V$.

Di seguito alcuni esempi di spazi vettoriali (si può verificare facilmente):

- $\mathbb{R}^n = \{(x_1, \dots, x_n) \mid x_i \in \mathbb{R} \forall i = 1, \dots, n\}$
- L'insieme di tutte le funzioni continue da $[a, b]$ a $\mathbb{R}$. $\forall x \in [a, b] \quad \forall \lambda \in \mathbb{R}$

Definizione 2.2 (Base). Una base di uno spazio vettoriale V è un insieme di vettori linearmente indipendenti che generano V .

$$\text{span}(v_1, \dots, v_n) = \{\lambda_1 v_1 + \dots + \lambda_n v_n \mid \lambda_i \in \mathbb{R} \forall i = 1, \dots, n\}$$

Ovvero l'insieme di tutte le combinazioni lineari dei vettori $v_1, \dots, v_n$. I vettori $v_1, \dots, v_n$ sono linearmente indipendenti se e solo se l'unica combinazione lineare nulla è quella banale, cioè:

$$\lambda_1 v_1 + \dots + \lambda_n v_n = 0 \implies \lambda_1 = \dots = \lambda_n = 0$$

Esempi di basi sono:

- $\underbrace{\{(1, 0, \dots, 0), (0, 1, \dots, 0), \dots, (0, 0, \dots, 1)\}}_{\text{(one hot)}}$ per $\mathbb{R}^n$ (base canonica)
- Base per tutte le funzioni $f : \mathbb{R} \rightarrow \mathbb{R}$:

$$f_1(x) = \begin{cases} 1 & x = x_1 \\ 0 & \text{altrimenti} \end{cases}$$

$$f_2(x) = \begin{cases} 1 & x = x_2 \\ 0 & \text{altrimenti} \end{cases}$$

$\vdots$

Definizione 2.3 (Dimensione). La dimensione di uno spazio vettoriale V è il numero di vettori in una base di V .

Definizione 2.4 (Mappe lineari (o trasformazioni lineari o omomorfismi)). Una mappa lineare $f : V \rightarrow W$ è una funzione che ha le seguenti proprietà:

$$\text{Additività} \quad T(u + v) = \underbrace{T(u) + T(v)}_{\in W} \quad \forall u, v \in V$$

Omogeneità $T(\lambda v) = \lambda T(v) \quad \forall v \in V, \forall \lambda \in \mathbb{R}$

Alcuni esempi di mappe lineari sono:

- L'identità $I : V \rightarrow V$ è una mappa lineare.
- La derivata $D : \mathcal{F}(\mathbb{R}) \rightarrow \mathcal{F}(\mathbb{R})$, definita come $D(f) = f'$ è una mappa lineare.
- L'integrazione $T : \mathcal{F}(\mathbb{R}) \rightarrow \mathbb{R}$, definita come $T(f) = \int_0^1 f(x)dx$ è una mappa lineare.
- Qualunque mappa $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ definita come:

$$T(x) = Ax \quad \rightarrow \quad T(x_1, \dots, x_n) = (a_{11}x_1 + \dots + a_{1n}x_n, \dots, a_{m1}x_1 + \dots + a_{mn}x_n)$$

dove A è una matrice $m \times n$, è una mappa lineare.

Osservazione 2.1. Le mappe lineari formano a loro volta uno spazio vettoriale.

Definizione 2.5 (Prodotto tra mappe lineari). Il prodotto tra mappe lineari $T : V \rightarrow W$ e $S : W \rightarrow U$ è la mappa lineare $S \circ T : V \rightarrow U$ definita come:

$$(S \circ T)(v) = S(T(v)) \quad \forall v \in V$$

proprietà dei prodotti tra mappe lineari:

$$\text{Associatività} \quad (S \circ T) \circ R = S \circ (T \circ R)$$

$$\text{Elemento neutro} \quad I \circ T = T \circ I = T$$

$$\text{Distributività} \quad (S + T) \circ R = S \circ R + T \circ R$$

Definizione 2.6 (Matrici). Data una mappa lineare $T : V \rightarrow W$, una base $v_1, \dots, v_n$ di V e una base $w_1, \dots, w_m$ di W , la matrice associata a T è l'array di valori in $\mathbb{R}$:

$$T = \begin{bmatrix} T_{1,1} & \dots & T_{1,n} \\ \vdots & \ddots & \vdots \\ T_{m,1} & \dots & T_{m,n} \end{bmatrix}$$

dove $T_{i,j}$ è il coefficiente della mappa lineare T associato al vettore v_j di V e al vettore w_i di W .

$$T_{i,j} = T_{1,j}w_1 + \dots + T_{m,j}w_m$$

Una matrice è la rappresentazione di una mappa lineare in una certa base.

Ricordiamo alcuni trucchi per il calcolo matriciale:

- Se la matrice A è simmetrica, allora $A = A^T$.
- $(BC)^T = C^T B^T$
- $(BC)^{-1} = C^{-1} B^{-1}$
- Se A è ortogonale, allora $A^T = A^{-1}$, quindi $A^T A = I$.

Definizione 2.7 (Norma di Frobenius). La norma (quadratica) di Frobenius di una matrice X è:

$$\|X\|_F^2 = \text{Tr}(X^T X) = \text{Tr}(X X^T)$$

dove Tr è la traccia della matrice.

Osservazione 2.2. Se X è un vettore, allora $\|X\|_F^2 = \text{Tr}(\underbrace{X X^T}_{n \times n}) = \text{Tr}(\underbrace{X^T X}_{1 \times 1}) = X^T X$.

Date A e B due matrici allora possiamo definire la norma di Frobenius come:

$$\|A - B\|_F^2 = \text{Tr}((A - B)^T (A - B)) = \text{Tr}(A^T A - 2A^T B + B^T B) = \text{Tr}(A^T A) + \text{Tr}(B^T B) - 2\text{Tr}(A^T B)$$

Questa si ottiene usando la linearità della traccia e la sua invarianza per le permutazioni cicliche.

Osservazione 2.3. Un vettore di soli uni può essere usato per manipolare facilmente le somme di matrici. Ad esempio, se si vuole sommare gli elementi di ogni riga di una matrice A si può calcolare:

$$A\mathbf{1}$$

Per sommare gli elementi di ogni colonna si può calcolare:

$$\mathbf{1}^T A = (A^T \mathbf{1})^T$$

Per sommare gli elementi di ogni riga e colonna si può calcolare:

$$\mathbf{1}^T A \mathbf{1}$$

Si osservi la seguente relazione:

$$x^T A x = \text{Tr}(x^T A x) = \text{Tr}(x x^T A) = \text{Tr}(X A)$$

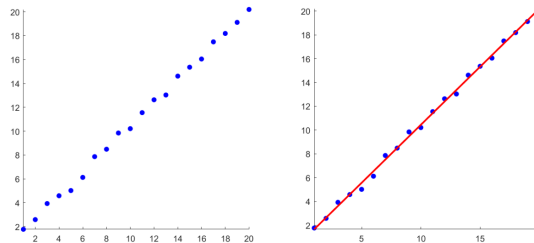
Definizione 2.8 (Prodotto scalare). Il prodotto scalare tra due vettori x e y è:

$$x^T y = \sum_{i=1}^n x_i y_i$$

3 Regressioni a dipendenza lineare

La regressione lineare è uno dei modelli statistici più semplici e fondamentali per analizzare la relazione tra una variabile dipendente (target) e una o più variabili indipendenti (feature). L'obiettivo della regressione lineare è trovare una funzione matematica che descriva al meglio questa relazione, permettendo di fare previsioni sui nuovi dati.

3.1 Regressione lineare



Qui possiamo immaginare che:

$$f_{\Theta}(x_i) = y_i$$

Per cui abbiamo il caso di un modello lineare, di parametri $\Theta = \{a, b\}$. Gli x_i prendono il nome di regressori. Per cui dati a, b possiamo mappare un output dato un nuovo input. Il criterio di errore che scegliamo è chiamato **loss** (o energy) function, e si indica con l :

$$l_{\Theta}(\{x_i, y_i\}) = \sum_{i=1}^n (y_i - f_{\Theta}(x_i))^2$$

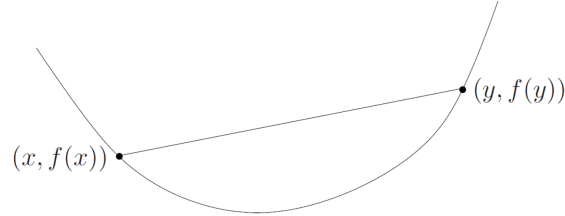
E questa vuole essere minimizzata rispetto a Θ .

Vediamo quindi il caso in cui f_{Θ} è lineare e l'errore l_{Θ} è quadratico. Il problema di ottimizzazione da risolvere è scegliere a e b in modo da minimizzare l'errore quadratico:

$$\epsilon = \min_{\Theta} l(\Theta)$$

In generale il metodo per ottimizzarla dipende un dipende dalle proprietà di l . Le funzioni che sono di nostro interesse sono quelle **Convesse**, per cui vale:

$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y) \quad \forall x, y \in \mathbb{R}, \forall \alpha \in (0, 1)$$



Assumeremo anche che f sia differenziabile, per cui possiamo calcolarne la derivata $\frac{df}{dx}$ per ogni x . In generale in ML avremo a che fare con funzioni di $n \gg 1$ parametri. Dobbiamo ora estendere il concetto di convessità a funzioni multivariate. La nozione di derivata può certamente essere estesa a gradiente:

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

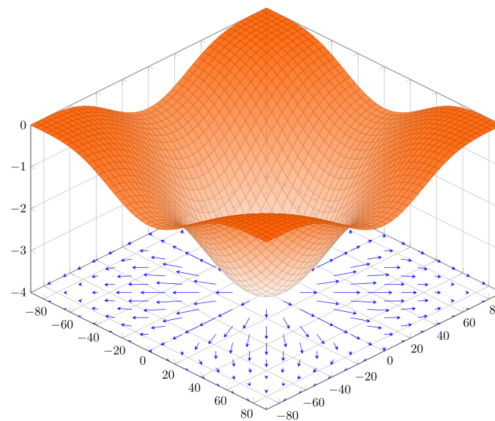
che è il vettore delle derivate parziali di f rispetto a ciascuna coordinata di $\mathbf{x}$. E la convessità si estende alle funzioni vettoriali:

$$f(\alpha \mathbf{x} + (1 - \alpha)\mathbf{y}) \leq \alpha f(\mathbf{x}) + (1 - \alpha)f(\mathbf{y}) \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^n, \forall \alpha \in (0, 1)$$

E la condizione globale ottimale è data da:

$$\nabla_{\mathbf{x}} f(\mathbf{x}^*) = 0 \implies f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \forall \mathbf{x} \in \mathbb{R}^n$$

Il gradiente codifica la direzione di massima variazione di f al punto $\mathbf{x}$, per cui il gradiente disegna un campo vettoriale



Lunghezza dei vettori Conosciamo la distanza euclidea tra due vettori:

$$d(a, b) = (|x_b - x_a|^2 + |y_b - y_a|^2)^{\frac{1}{2}}$$

Tuttavia possiamo generalizzare questa nozione a:

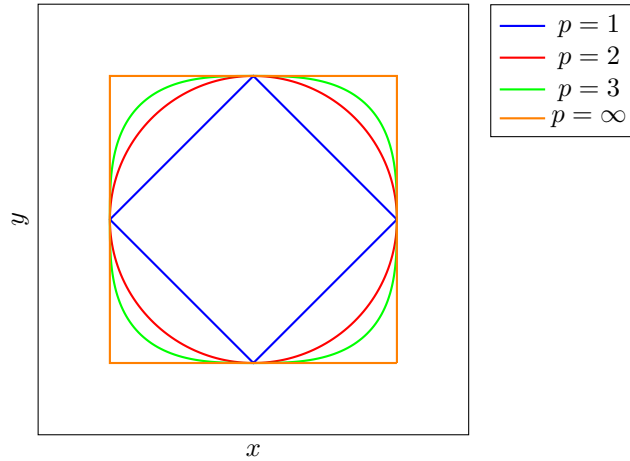
$$\|x - y\|_p = (|x_1 - y_1|^p + |x_2 - y_2|^p)^{\frac{1}{p}}$$

per ogni $p \geq 1$. E possiamo anche generalizzare da $\mathbb{R}^2$ a $\mathbb{R}^k$:

$$\|\mathbf{x} - \mathbf{y}\|_p = \left(\sum_{i=1}^k |x_i - y_i|^p \right)^{1/p}$$

La norma di un vettore è la distanza tra il vettore e l'origine:

$$\|\mathbf{x}\|_p = \|\mathbf{x} - \mathbf{0}\|_p$$



Vediamo quindi come possiamo trovare una soluzione per la regressione lineare, ovvero:

$$\Theta^* = \arg \min_{\Theta \in \mathbb{R}^2} l(\Theta)$$

dove $l : \mathbb{R}^2 \rightarrow \mathbb{R}$ è definita come:

$$l(a, b) = \sum_{i=1}^n (y_i - ax_i - b)^2$$

La soluzione può essere trovata ponendo $\nabla_{\Theta} l(\Theta) = 0$:

$$\begin{aligned} \nabla_{\Theta} \sum_{i=1}^n (y_i - ax_i - b)^2 &= \sum_{i=1}^n \nabla_{\Theta} (y_i - ax_i - b)^2 \\ &= \sum_{i=1}^n \nabla_{\Theta} [y_i^2 - 2ax_i y_i - 2by_i + a^2 x_i^2 + 2abx_i + b^2] \\ &= \begin{bmatrix} \sum_{i=1}^n (2ax_i^2 + 2bx_i - 2x_i y_i) \\ \sum_{i=1}^n (2ax_i + 2b - 2y_i) \end{bmatrix} \\ &= 2 \begin{bmatrix} \sum_{i=1}^n (ax_i^2 + bx_i - x_i y_i) \\ \sum_{i=1}^n (ax_i + b - y_i) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \end{aligned}$$

Possiamo utilizzare una forma matriciale dato che il modello di apprendimento per la regressione lineare è lineare rispetto ai parametri (non lineare a x visto il bias). Quindi $y_i = ax_i + b$ possiamo riscriverlo come:

$$\underbrace{\begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}}_{\mathbf{y}} = \underbrace{\begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix}}_{\mathbf{X}} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{\Theta}$$

L'errore quadratico si può quindi riscrivere come:

$$l(\Theta) = \|\mathbf{y} - \mathbf{X}\Theta\|_2^2 = (\mathbf{y} - \mathbf{X}\Theta)^T(\mathbf{y} - \mathbf{X}\Theta) = \mathbf{y}^T\mathbf{y} - 2\mathbf{y}^T\mathbf{X}\Theta + \Theta^T\mathbf{X}^T\mathbf{X}\Theta$$

e il gradiente rispetto a Θ di $l(\Theta)$ è:

$$-2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\Theta = 0$$

da cui si ottiene:

$$\mathbf{X}^T\mathbf{X}\Theta = \mathbf{X}^T\mathbf{y}$$

e quindi:

$$\Theta = \underbrace{(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T}_{\text{pseudo-inversa di } \mathbf{X}} \mathbf{y}$$

questa è detta **equazione normale**. Indichiamo con $\mathbf{X}^\dagger$ la pseudo-inversa di $\mathbf{X}$. Osserviamo che se $\mathbf{X}$ è una matrice quadrata e invertibile, allora $\mathbf{X}^\dagger = \mathbf{X}^{-1}$. Vediamo come abbiamo ottenuto il fatto che

$$\mathbf{y}^T\mathbf{y} - 2\mathbf{y}^T\mathbf{X}\Theta + \Theta^T\mathbf{X}^T\mathbf{X}\Theta \xrightarrow{\nabla_{\Theta}} -2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\Theta$$

Poniamo $f(\Theta) = \Theta^T\mathbf{A}\Theta$. Allora:

$$\begin{aligned} \nabla_{\Theta} f(\Theta) &= \nabla_{\Theta}(\Theta_1 \dots \Theta_n) \begin{bmatrix} \mathbf{A}_{1,1} & \dots & \mathbf{A}_{1,n} \\ \vdots & \ddots & \vdots \\ \mathbf{A}_{n,1} & \dots & \mathbf{A}_{n,n} \end{bmatrix} \begin{bmatrix} \Theta_1 \\ \vdots \\ \Theta_n \end{bmatrix} = \\ &= \nabla_{\Theta} \sum_{i=1}^n \sum_{j=1}^n \Theta_i \mathbf{A}_{i,j} \Theta_j = \\ &= \begin{bmatrix} \frac{\partial}{\partial \Theta_1} \sum_{i=1}^n \sum_{j=1}^n \Theta_i \mathbf{A}_{i,j} \Theta_j \\ \vdots \\ \frac{\partial}{\partial \Theta_n} \sum_{i=1}^n \sum_{j=1}^n \Theta_i \mathbf{A}_{i,j} \Theta_j \end{bmatrix} = \\ &= \begin{bmatrix} \sum_{j=1}^n \mathbf{A}_{1,j} \Theta_j + \sum_{i=1}^n \sum_{j=1}^n \mathbf{A}_{i,j} \Theta_j \\ \vdots \\ \sum_{j=1}^n \mathbf{A}_{n,j} \Theta_j + \sum_{i=1}^n \sum_{j=1}^n \mathbf{A}_{i,j} \Theta_j \end{bmatrix} = \\ &= \begin{bmatrix} \sum_{i=1}^n (\mathbf{A}_{1,i} + \mathbf{A}_{i,1}) \Theta_i \\ \vdots \\ \sum_{i=1}^n (\mathbf{A}_{n,i} + \mathbf{A}_{i,n}) \Theta_i \end{bmatrix} = \\ &= (\mathbf{A} + \mathbf{A}^T)\Theta \end{aligned}$$

Si osserva che se $\mathbf{A}$ è simmetrica, allora:

$$\nabla_{\Theta} f(\Theta) = 2\mathbf{A}\Theta$$

Nel caso più generale di dimensioni maggiori possiamo generalizzare $y_i = a_i x_i + b_i$ come:

$$\mathbf{y}_i = \mathbf{A}\mathbf{x}_i + \mathbf{b}$$

Definendo le matrici:

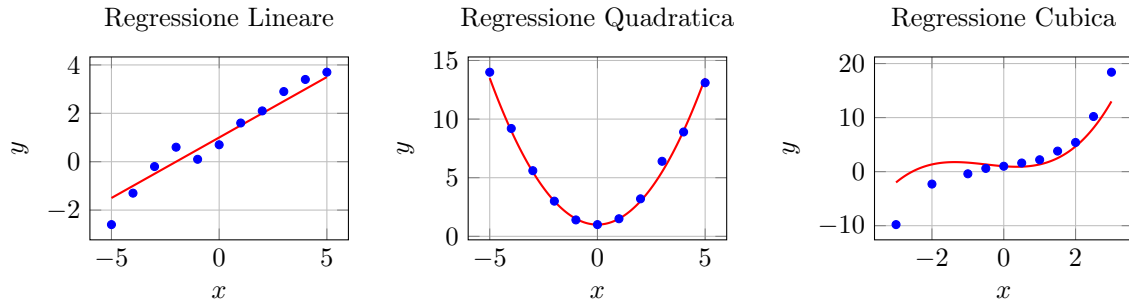
$$\mathbf{X} = \begin{bmatrix} \left| \right| \left| \right| \vdots \left| \right| \\ \mathbf{x}_1 \quad \mathbf{x}_2 \quad \vdots \quad \mathbf{x}_n \\ \left| \right| \left| \right| \vdots \left| \right| \\ 1 \quad 1 \quad \vdots \quad 1 \end{bmatrix} \quad \mathbf{Y} = \begin{bmatrix} \left| \right| \left| \right| \vdots \left| \right| \\ \mathbf{y}_1 \quad \mathbf{y}_2 \quad \vdots \quad \mathbf{y}_n \\ \left| \right| \left| \right| \vdots \left| \right| \end{bmatrix} \quad \Theta = \begin{bmatrix} \mathbf{A} \\ \mathbf{b}^T \end{bmatrix} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \\ b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

Abbiamo una forma chiusa per la soluzione a $\nabla_{\Theta} l(\Theta) = 0$:

$$\Theta = (\mathbf{X}\mathbf{X}^T)^{-1}\mathbf{X}\mathbf{Y}^T$$

3.2 Modelli di regressione polinomiale (regressione lineare con dati polinomiali)

Un modello polinomiale nei dati rimane lineare nei parametri.



Si hanno equazioni del tipo:

$$y_i = b + \sum_{j=1}^k a_j x_i^j \quad \forall i = 1, \dots, n$$

Nella notazione matriciale:

$$\underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}}_{\mathbf{y}} = \underbrace{\begin{bmatrix} x_1^k & x_1^{k-1} & \dots & x_1 & 1 \\ x_2^k & x_2^{k-1} & \dots & x_2 & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_n^k & x_n^{k-1} & \dots & x_n & 1 \end{bmatrix}}_{\mathbf{X}} \underbrace{\begin{bmatrix} a_k \\ a_{k-1} \\ \vdots \\ a_1 \\ b \end{bmatrix}}_{\Theta}$$

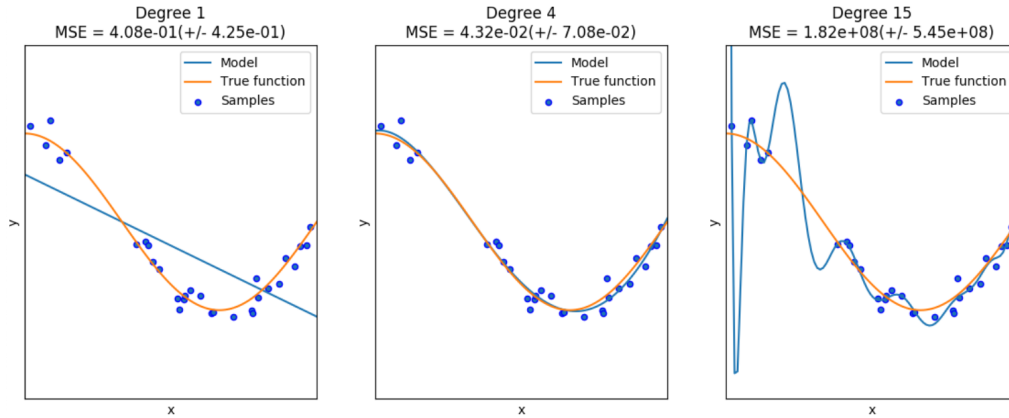
Dove ogni riga di $\mathbf{X}$ corrisponde a una feature polinomiale, e ogni potenza può essere considerata come una feature a sè stante.

Ricordiamo il teorema di **Stone-Weierstrass**:

Teorema 3.1 (Stone-Weierstrass). *Sia $f : [a, b] \rightarrow \mathbb{R}$ una funzione continua. Allora per ogni $\epsilon > 0$ esiste un polinomio $p : [a, b] \rightarrow \mathbb{R}$ tale che:*

$$|f(x) - p(x)| < \epsilon \quad \forall x \in [a, b]$$

Questo teorema ci assicura che possiamo approssimare una funzione continua con un polinomio, e quindi possiamo approssimare una funzione polinomiale con un polinomio. Per cui aumentare di grado il polinomio effettivamente migliora l'approssimazione. Tuttavia, se il polinomio è troppo complesso, possiamo incorrere in **overfitting** (in cui si fitta anche il rumore).



Definiamo:

- **Overfitting:** quando il modello si adatta troppo ai dati, imparando anche il rumore.
- **Underfitting:** quando il modello non è sufficientemente in grado di descrivere i dati.

Un modo per evitare l'overfitting è di utilizzare una parte dei dati per il **training** e una parte per la **validation**. Si potrebbe anche utilizzare una **n -fold cross-validation**, ovvero dividere i dati in k parti e utilizzare una parte per il training e le altre $k - 1$ per la validation, iterando questo processo k volte.

3.3 Regressione logistica

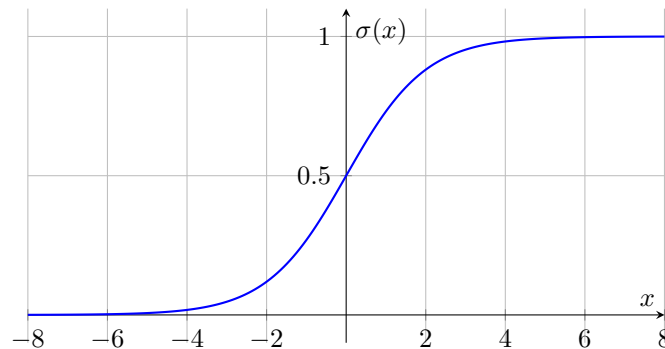
I problemi di regressione polinomiale hanno un output che è continuo, ovvero può assumere qualsiasi valore in un intervallo. Tuttavia, spesso si ha a che fare con problemi di classificazione, ovvero si vuole classificare un'osservazione in due (binaria) o più classi discrete.

Nel caso dei problemi di classificazione binaria, vogliamo una funzione che restituisca un valore compreso tra 0 e 1, che possiamo interpretare come la probabilità di appartenenza a una certa classe.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Questa funzione è detta **sigmoide**

Funzione Sigmoidale



La nostra **loss function** sarà (MSE):

$$l(\{x_i, y_i\}) = \sum_{i=1}^n (y_i - \sigma(ax_i + b))^2$$

Questo modello non è lineare. Questa funzione non è convessa in a, b , ma possiamo renderla convessa usando la **log loss**:

$$l(\{x_i, y_i\}) = \sum_{i=1}^n (c(x_i, y_i))$$

Dove:

$$c(x_i, y_i) = \begin{cases} -\ln(\sigma(ax_i + b)) & \text{se } y_i = 1 \\ -\ln(1 - \sigma(ax_i + b)) & \text{se } y_i = 0 \end{cases} = -y_i \ln(\sigma(ax_i + b)) - (1 - y_i) \ln(1 - \sigma(ax_i + b))$$

Questa funzione è convessa (si può dimostrare) e differenziabile. Viene chiamata **cross-entropy**. Possiamo quindi calcolare il gradiente rispetto a $\Theta = \{a, b\}$:

$$\nabla_{\Theta} \sum_{i=1}^n y_i \ln(\sigma(ax_i + b)) + (1 - y_i) \ln(1 - \sigma(ax_i + b)) = 0$$

Consideriamo il gradiente di ciascun termine della sommatoria

$$\begin{aligned} & \nabla_{\Theta} y_i \ln(\sigma(ax_i + b)) + (1 - y_i) \ln(1 - \sigma(ax_i + b)) = \\ & \nabla_{\Theta} y_i \ln(\sigma(ax_i + b)) + \nabla_{\Theta} (1 - y_i) \ln(1 - \sigma(ax_i + b)) = \\ & y_i \nabla_{\Theta} \underbrace{\ln(\sigma(ax_i + b))}_{f(g(h(\Theta)))} + (1 - y_i) \nabla_{\Theta} \ln(1 - \sigma(ax_i + b)) = \end{aligned}$$

Possiamo dunque applicare la regola della catena:

$$\begin{aligned} \frac{\partial}{\partial a} f(g(h(a, b))) &= \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial h} \cdot \frac{\partial h}{\partial a} = \\ &= \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial h} \cdot \frac{\partial}{\partial a} ax_i + b = \\ &= \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial h} \cdot x_i = \\ &= \frac{\partial f}{\partial g} \cdot \frac{\partial \sigma(ax_i + b)}{\partial (ax_i + b)} \cdot x_i = \\ &= \frac{\partial f}{\partial g} \cdot \frac{\partial}{\partial (ax_i + b)} \frac{1}{1 + e^{-(ax_i + b)}} \cdot x_i = \\ &= \frac{\partial f}{\partial g} \cdot \frac{e^{-(ax_i + b)}}{(1 + e^{-(ax_i + b)})^2} \cdot x_i = \\ &= \frac{\partial f}{\partial g} \cdot \frac{1}{1 + e^{-(ax_i + b)}} \cdot \frac{e^{-(ax_i + b)}}{1 + e^{-(ax_i + b)}} \cdot x_i = \\ &= \frac{\partial f}{\partial g} \cdot \frac{1}{1 + e^{-(ax_i + b)}} \cdot \frac{(1 + e^{-(ax_i + b)}) - 1}{1 + e^{-(ax_i + b)}} \cdot x_i = \\ &= \frac{\partial f}{\partial g} \cdot \frac{1}{1 + e^{-(ax_i + b)}} \cdot \left(1 - \frac{1}{1 + e^{-(ax_i + b)}}\right) \cdot x_i = \\ &= \frac{\partial f}{\partial g} \cdot \sigma(ax_i + b) \cdot (1 - \sigma(ax_i + b)) \cdot x_i = \\ &= \frac{\partial \ln(\sigma(ax_i + b))}{\partial \sigma(ax_i + b)} \cdot \sigma(ax_i + b) \cdot (1 - \sigma(ax_i + b)) \cdot x_i = \\ &= \frac{1}{\sigma(ax_i + b)} \cdot \sigma(ax_i + b) \cdot (1 - \sigma(ax_i + b)) \cdot x_i = \\ &= (1 - \sigma(ax_i + b)) \cdot x_i \end{aligned}$$

E si prosegue in maniera analoga per il termine b .

Guardando la derivata di $\ln(\sigma(ax_i + b))$ possiamo vedere che:

$$\frac{\partial}{\partial a} \ln(\sigma(ax_i + b)) = (1 - \sigma(ax_i + b))x_i$$

per cui:

- $\nabla l_{\Theta} = 0$ non è un sistema lineare
- $\nabla l_{\Theta} = 0$ è un'equazione **trascendente**, per cui non possiamo trovare una soluzione analitica

3.4 Sistema sottodeterminato - regolarizzazione

Se abbiamo n equazioni linearmente dipendenti e $m > n$ parametri ($\mathbf{A}$ è larga). L'idea generale è fare assunzioni addizionali e scriverle come nuovi termini. Questo processo ha alcuni benefici.

- Riduce il numero di dati necessari
- Fare un'ottimizzazione migliore

Regolarizzazione di Tikhonov

Aggiungiamo un termine di regolarizzazione L_2

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \alpha \|\mathbf{x}\|_2^2$$

Questa "penalty" penalizza in particolare i valori superiori a 1 di $\mathbf{x}$. Il problema rimane convesso e differenziabile:

$$\begin{aligned} \nabla_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \alpha \|\mathbf{x}\|_2^2 &= 0 \\ \nabla_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \alpha \nabla_{\mathbf{x}} \|\mathbf{x}\|_2^2 &= 0 \\ 2\mathbf{A}^T \mathbf{Ax} - 2\mathbf{A}^T \mathbf{b} + 2\alpha \mathbf{x} &= 0 \\ \mathbf{A}^T \mathbf{Ax} - \mathbf{A}^T \mathbf{b} + \alpha \mathbf{x} &= 0 \\ \mathbf{A}^T \mathbf{Ax} + \alpha \mathbf{x} &= \mathbf{A}^T \mathbf{b} \\ (\mathbf{A}^T \mathbf{A} + \alpha \mathbf{I})\mathbf{x} &= \mathbf{A}^T \mathbf{b} \end{aligned}$$

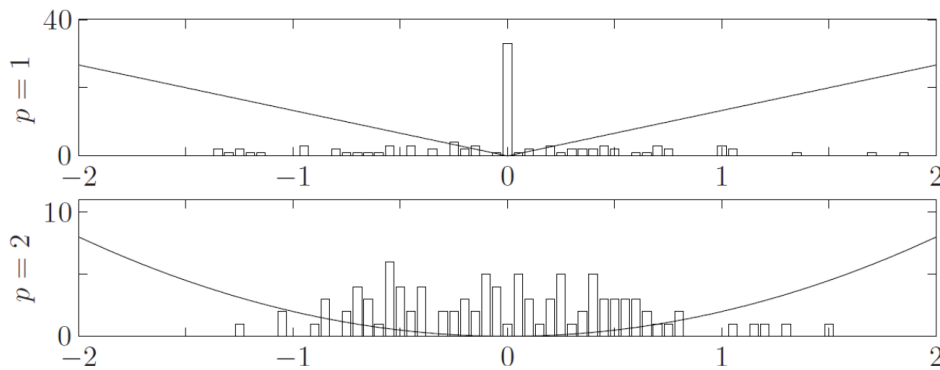
Praticamente stiamo aggiungendo $\alpha \mathbf{I}$ alla matrice $\mathbf{A}^T \mathbf{A}$.

Sparsità

Se un vettore $\mathbf{x}$ ha molti elementi pari a 0 (più della metà delle componenti sono 0), diciamo che $\mathbf{x}$ è **sperso**. Con la regolarizzazione di Tikhonov abbiamo:

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \alpha \|\mathbf{x}\|_p^p$$

Osserviamo che a seconda del valore di p penalizziamo in maniera differente i valori di $\mathbf{x}$, con la norma di 1 che penalizza linearmente tutti i valori e la norma di 2 che penalizza valori un po' più grandi.



Questo effetto si dice **shrinkage**.

Osservazione 3.1. Osserviamo dunque che la regolarizzazione di Tikhonov con norma $p = 1$ implica che la soluzione sia sparsa.

In generale la regolarizzazione si può indicare:

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_p^p + \alpha \rho(\mathbf{x})$$

Dove $\rho(\mathbf{x})$ è una funzione che penalizza la complessità del modello.

Si parla di sparsità anche quando la matrice $\mathbf{A}$ è sparsa.

Smoothing

Possiamo anche usare la regolarizzazione per **smootheare** i dati. In questo caso possiamo considerare $\|\mathbf{Dx}\|$ come una penalità, dove $\mathbf{D}$ è un operatore di differenziazione. Per cui:

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \alpha \|\mathbf{Dx}\|_2^2$$

Per preservare salti occasionali possiamo usare la norma L_1 :

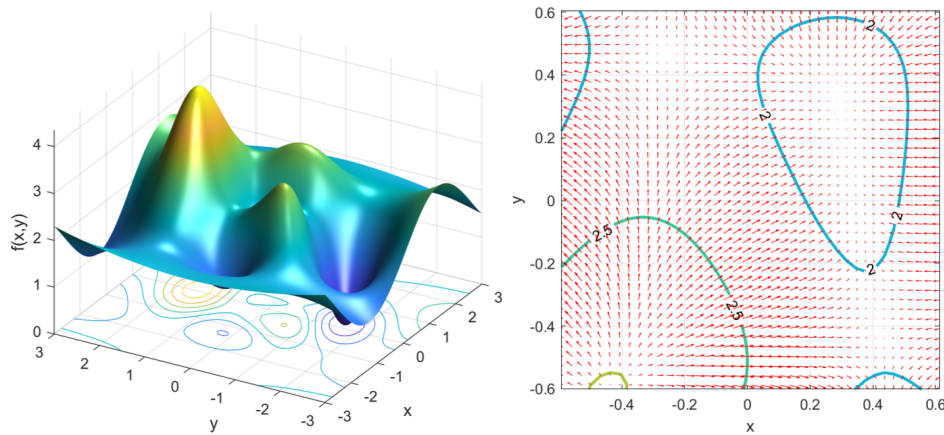
$$\|\mathbf{Dx}\|_1 = \sum_{i=1}^{n-1} |x_{i+1} - x_i|$$

4 Discesa del gradiente

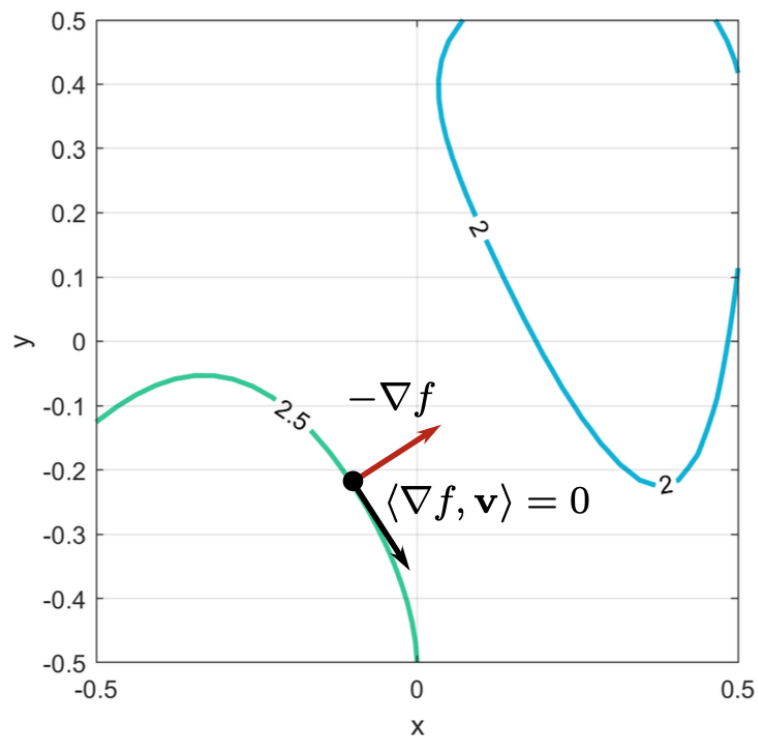
La discesa del gradiente è un metodo iterativo per trovare il minimo di una funzione. L'idea è di partire da un punto iniziale $\mathbf{x}^{(0)}$ e di calcolare iterativamente:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \alpha \nabla f(\mathbf{x}^{(t)})$$

dove α è un parametro che rappresenta la dimensione del passo.



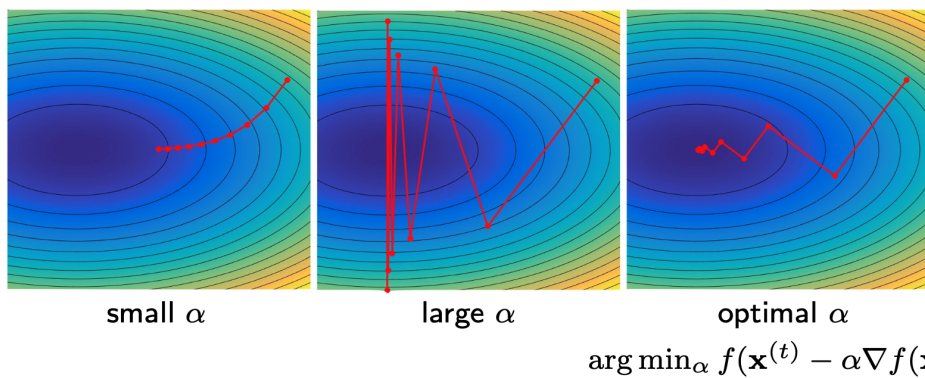
Notiamo che il gradiente è ortogonale alla curva di livello.



La discesa del gradiente si ferma quando arriviamo in un punto stazionario, ed è molto sensibile ai valori iniziali. Il punto è che non sempre i punti stazionari sono minimi locali. Il parametro α indica il **learning rate**. La lunghezza del passo corrisponde a:

$$\alpha \|\nabla f\|$$

Se α è troppo piccolo potremmo convergere troppo lentamente, se è troppo grande potremmo non convergere affatto, o addirittura divergere. Il parametro α può essere scelto in maniera ottimale con algoritmi euristici (line search).



Per il learning rate possiamo seguire una schedule per fare in modo che la α si adatti alla discesa. Si può introdurre una componente di **momentum** per pesare la direzione di discesa e accumulare gradienti passati.

$$\begin{aligned}\mathbf{v}^{(t+1)} &= \lambda \mathbf{v}^{(t)} - \alpha \nabla f(\mathbf{x}^{(t)}) \\ \mathbf{x}^{(t+1)} &= \mathbf{x}^{(t)} + \mathbf{v}^{(t+1)}\end{aligned}$$

Quindi i calcoli per lo SGD senza momentum sarebbero:

$$\begin{aligned}\mathbf{x}^{(1)} &= \mathbf{x}^{(0)} - \alpha \nabla f(\mathbf{x}^{(0)}) \\ \mathbf{x}^{(2)} &= \mathbf{x}^{(1)} - \alpha \nabla f(\mathbf{x}^{(1)}) \\ &= \mathbf{x}^{(0)} - \alpha \nabla f(\mathbf{x}^{(0)}) - \alpha \nabla f(\mathbf{x}^{(1)}) \\ &\vdots \\ \mathbf{x}^{(t+1)} &= \mathbf{x}^{(0)} - \alpha \sum_{i=0}^t \nabla f(\mathbf{x}^{(i)})\end{aligned}$$

Mentre con momentum sarebbe:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(0)} - \alpha \sum_{i=0}^t \frac{1 - \lambda^{t+1-i}}{1 - \lambda} \nabla f(\mathbf{x}^{(i)})$$

In forma più generale:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(0)} + \alpha \sum_{i=0}^t \Gamma_i^t \nabla f(\mathbf{x}^{(i)})$$

per qualche matrice diagonale Γ_i . La discesa del gradiente può essere applicata a problemi non convessi e pure a problemi convessi (sia regressione lineare che logistica). Si ottengono soluzioni più efficienti e stabili.

4.1 Stochastic Gradient Descent

Il gradient descent stocastico calcola il gradiente della **loss** in meno dati rispetto alla totalità dei dati che considera la discesa del gradiente in generale. Quindi invece di n dati, posso considerare $m \ll n$ dati e ottenere:

$$\frac{1}{m} \sum_{i=1}^m \nabla \hat{l}(\theta)(\mathcal{B}) \approx \frac{1}{n} \sum_{i=1}^n \nabla \hat{l}(\theta)(\mathcal{T})$$

dove $\mathcal{B} \subset \mathcal{T}$ e $\mathcal{T} = \{x_i, y_i\}$. La soluzione del sottoinsieme di dati che consideriamo si dice **mini-batch**.

L'algoritmo si sviluppa in questo modo:

1. Inizializziamo θ
2. Scegliamo un mini-batch $\mathcal{B}$
3. Aggiorniamo θ con il passo del gradiente:

$$\theta \leftarrow \theta - \alpha \nabla l_{\theta}(\mathcal{B})$$

4. Ritorno al punto (2)

Quando gli step (2) – (4) sono stati eseguiti per tutti i mini-batch, diciamo che è stato compiuto un **epoch**. Un algoritmo può procedere per molte epoche.

5 Multi-Layer Perceptron & Backpropagation

In deep learning si utilizzano modelli con molti parametri, e questi sono detti modelli **deep**. Possiamo comporre funzioni non lineari e lineari in maniera iterativa per ottenere funzioni molto complesse.

Composizione di funzioni lineari:

$$f \circ f(\mathbf{x})$$

Questa funzione rimane lineare (prodotto di matrici non è quadratico, ma va visto come una trasformazione lineare). Data la funzione logistica σ possiamo comporre le funzioni e ottenere una funzione non lineare:

$$\sigma \circ f(\mathbf{x})$$

E possiamo considerare layer multipli di questo tipo:

$$\text{output} \leftarrow \underbrace{(\sigma \circ \sigma)}_{\text{output layer}} \circ \underbrace{(\sigma \circ \sigma)}_{\text{layer } n-1} \circ \cdots \circ \underbrace{(\sigma \circ \sigma)}_{\text{input layer}} (\mathbf{x}) \leftarrow \text{input}$$

In generale possiamo scegliere la funzione σ in maniera arbitraria:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \text{o} \quad \sigma(x) = \max\{0, x\} \text{ (rectified linear unit - ReLU)}$$

Questo tipo di rete neurale è detta **Multi-Layer Perceptron** (MLP). I parametri di tutto il modello sono distribuiti tra i vari layer.

$$g_{\Theta}(\mathbf{x}) = (\sigma \circ f_{\Theta_n} \circ (\sigma \circ f_{\Theta_{n-1}}) \circ \cdots \circ (\sigma \circ f_{\Theta_1}))(\mathbf{x})$$

Ogni layer di output ha una rappresentazione nascosta dove vengono memorizzati i pesi in una matrice $\mathbf{W}_l$ e i bias in un vettore $\mathbf{b}_l$.

$$\mathbf{x}_{l+1} = \sigma_l(\mathbf{W}_l \mathbf{x}_l + \mathbf{b}_l)$$

Ogni riga è detta **neurone** (o unità nascosta):

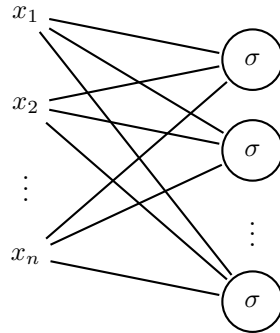
$$\mathbf{W}\mathbf{x} = \begin{pmatrix} \text{---} & \text{neuron} & \text{---} \\ & \vdots & \\ \text{---} & \text{neuron} & \text{---} \end{pmatrix} \begin{pmatrix} | \\ \mathbf{x} \\ | \end{pmatrix}$$

Parliamo di larghezza di layer per indicare il numero di neuroni in ogni layer. Possiamo considerare due interpretazioni:

- Ogni layer è funzione vettore-vettore $\mathbb{R}^p \rightarrow \mathbb{R}^q$
- Ogni layer ha q neuroni che stanno agendo in parallelo. Ogni neurone si comporta come una funzione scalare $\mathbb{R}^p \rightarrow \mathbb{R}$

Ogni layer appare come:

$$\sigma(\mathbf{W}\mathbf{x}) = \sigma \circ \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \sigma \circ \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$



Osservazione 5.1. *Il layer di output determina il codominio della rete. Se ad esempio σ è la funzione logistica allora la rete mapperà:*

$$\mathbb{R}^p \rightarrow [0, 1]^q$$

Teorema 5.1 (Teorema di approssimazione universale). *Per ogni insieme compatto $\Omega \subset \mathbb{R}^p$, lo spazio delle funzioni $\phi = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ è denso in $C(\Omega)$ (σ è la funzione logistica). Per cui per qualunque funzione continua e per ogni $\epsilon > 0$ esiste un $q \in \mathbb{N}$ e dei pesi tali che:*

$$\left| f(\mathbf{x}) - \sum_{k=1}^q u_k \phi_k(\mathbf{x}) \right| \leq \epsilon \quad \forall \mathbf{x} \in \Omega$$

Dati quindi una serie di parametri $\{\mathbf{x}_i, \mathbf{y}_i\}$ per

$$g_{\Theta}(\mathbf{x}_i) = (\sigma \circ f_{\Theta_n} \circ (\sigma \circ f_{\Theta_{n-1}}) \circ \dots \circ (\sigma \circ f_{\Theta_1}))(\mathbf{x}_i)$$

e possiamo considerare la loss MSE:

$$l_{\Theta} = \frac{1}{n} \sum_{i=1}^n \|\mathbf{y}_i - g_{\Theta}(\mathbf{x}_i)\|_2^2$$

e dobbiamo dunque calcolarne il gradiente:

$$\nabla l_{\Theta}(\{\mathbf{x}_i, \mathbf{y}_i\}) = \frac{1}{n} \sum_{i=1}^n \nabla_{\Theta} \|\mathbf{y}_i - g_{\Theta}(\mathbf{x}_i)\|_2^2$$

Questo calcolo è molto oneroso computazionalmente, ma esistono dei modi per ottimizzare questo calcolo.

5.1 Grafi computazionali

Consideriamo una funzione $f : \mathbb{R} \rightarrow \mathbb{R}$. Un grafo computazionale è un grafo diretto in cui ogni nodo rappresenta una variabile e ogni arco rappresenta un'operazione.

Esempio 5.1. $f(x) = \log x + \sqrt{\log x}$

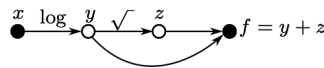
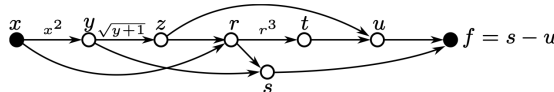


Figura 1: Grafo computazionale di $f(x) = \log x + \sqrt{\log x}$

Osserviamo che il grafo viene percorso da sinistra a destra (traversata in avanti). Inoltre va detto che esistono più grafi computazionali per la stessa funzione.

Esempio 5.2. $f(x) = \frac{\log(x + \sqrt{x^2 + 1})}{x^2} - \frac{\log^3(x + \sqrt{x^2 + 1})}{\sqrt{x^2 + 1}}$



Useremo adesso i grafi computazionali per computare il gradiente di una funzione (loss), che avranno sempre un nodo di output e molti nodi di input (# weights). Riconsideriamo la funzione $f(x) = \log x + \sqrt{\log x}$ di cui è riportato il grafo computazionale1.

1. **Differenziazione automatica - Forward mode** Si va da sinistra a destra calcolando le derivate dei nodi intermedi rispetto ai loro input

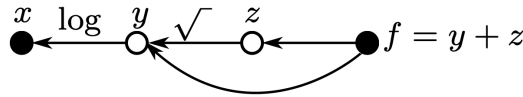
$$\begin{aligned}\frac{\partial x}{\partial x} &= 1 \\ \frac{\partial y}{\partial x} &= \frac{\partial y}{\partial x} \frac{\partial x}{\partial x} = \frac{\partial \log x}{\partial x} \frac{\partial x}{\partial x} = \frac{1}{x} \frac{\partial x}{\partial x} = \frac{1}{x} \\ \frac{\partial z}{\partial x} &= \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial \sqrt{y}}{\partial y} \frac{\partial y}{\partial x} = \frac{1}{2\sqrt{y}} \frac{\partial y}{\partial x} \\ \frac{\partial f}{\partial x} &= \frac{\partial f}{\partial y} \frac{\partial y}{\partial x} + \frac{\partial f}{\partial z} \frac{\partial z}{\partial x} = \frac{\partial(y+z)}{\partial y} \frac{\partial y}{\partial x} + \frac{\partial(y+z)}{\partial z} \frac{\partial z}{\partial x}\end{aligned}$$

Sia quando abbiamo computato $f(x)$ sia per il calcolo di $\frac{\partial f}{\partial x}(x)$ abbiamo attraversato il grafo in avanti, per cui il costo computazionale non è differente. Tuttavia se l'input è multi-dimensionale avremmo che:

$$\text{cost of } \nabla f(\mathbf{x}) = p \times \text{cost of } f(\mathbf{x}) \quad f : \mathbb{R}^p \rightarrow \mathbb{R}$$

Notiamo che la differenziazione automatica **non** è come applicare la differenziazione simbolica, dato che stiamo accumulando valori durante l'esecuzione della differenziazione automatica per ottenere valori numerici invece di espressioni.

2. **Differenziazione automatica - Reverse mode** Si va da destra a sinistra calcolando le derivate $\frac{\partial f}{\partial z}, \dots, \frac{\partial f}{\partial x}$ rispetto ai nodi intermedi quindi ora consideriamo:



$$\begin{aligned}\frac{\partial f}{\partial f} &= 1 \\ \frac{\partial f}{\partial z} &= \frac{\partial f}{\partial f} \frac{\partial f}{\partial z} = \frac{\partial f}{\partial f} \frac{\partial(y+z)}{\partial z} = \frac{\partial f}{\partial f} \\ \frac{\partial f}{\partial y} &= \frac{\partial f}{\partial z} \frac{\partial z}{\partial y} + \frac{\partial f}{\partial f} \frac{\partial f}{\partial y} = \frac{\partial f}{\partial z} \frac{\partial \sqrt{y}}{\partial y} + \frac{\partial f}{\partial f} \frac{\partial(y+z)}{\partial y} = \frac{\partial f}{\partial z} \frac{1}{2\sqrt{y}} + \frac{\partial f}{\partial f} \\ \frac{\partial f}{\partial x} &= \frac{\partial f}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial f}{\partial y} \frac{\partial \log x}{\partial x} = \frac{\partial f}{\partial y} \frac{1}{x}\end{aligned}$$

Osservazione 5.2. Per valutare i nodi intermedi $y, z, \dots$ dobbiamo aver attraversato il grafo in avanti (per calcolare f). Quindi avremo un **forward pass** e poi i **backward pass** per calcolare le derivate.

Osservazione 5.3. *Forward e backward pass* si utilizzano a seconda del compito da svolgere. Generalmente utilizziamo **backward pass** per modelli che hanno più output rispetto agli input, mentre **forward pass** per modelli che hanno più input rispetto agli output. Ci converrà spesso utilizzare un approccio backward.

Si dice **backpropagation** la tecnica di differenziazione automatica in **reverse mode** applicata alla rete neurale.

6 Principal Component Analysis

6.1 Principal Components

Sia $\mathbf{X} = [\mathbf{x}_1 \dots \mathbf{x}_n] \in \mathbb{R}^{d \times n}$ la matrice dei dati *centrati* (ogni colonna è un campione, la media di riga è zero). L'obiettivo è individuare un vettore unitario $\mathbf{v} \in \mathbb{R}^d$ (*asse principale*) lungo il quale

la proiezione dei dati possieda la massima varianza possibile, oppure, in maniera equivalente, che minimizzi l'errore di ricostruzione dopo la proiezione lungo tale vettore. Formalmente:

$$\min_{\|\mathbf{v}\|_2=1} \sum_{i=1}^n \|\mathbf{x}_i - \text{proj}_{\mathbf{v}} \mathbf{x}_i\|_2^2, \quad \text{proj}_{\mathbf{v}} \mathbf{x} = \frac{\mathbf{x}^\top \mathbf{v}}{\mathbf{v}^\top \mathbf{v}} \mathbf{v}.$$

Sviluppando il quadrato e sfruttando la centrazione dei dati, il problema si riscrive come

$$\min_{\|\mathbf{v}\|_2=1} (-\|\mathbf{X}^\top \mathbf{v}\|_2^2) = \max_{\|\mathbf{v}\|_2=1} \mathbf{v}^\top \mathbf{X} \mathbf{X}^\top \mathbf{v}.$$

La matrice $\mathbf{S} = \frac{1}{n} \mathbf{X} \mathbf{X}^\top$ è la *covarianza* campionaria ed è simmetrica semidefinita positiva. Il termine al numeratore della frazione di Rayleigh $\mathbf{v}^\top \mathbf{S} \mathbf{v}$ assume il valore massimo proprio sull'autovettore $\mathbf{u}_1$ associato all'autovalore dominante λ_1 . Di conseguenza

$$\mathbf{u}_1 = \arg \max_{\|\mathbf{v}\|_2=1} \mathbf{v}^\top \mathbf{S} \mathbf{v}$$

è la **prima componente principale**. I coefficienti $z_{i1} = \mathbf{u}_1^\top \mathbf{x}_i$ (prime coordinate latenti o *scores*) soddisfano

$$\text{Var}(z_{i1}) = \lambda_1 \implies \text{la frazione di varianza spiegata è } \lambda_1 / (\text{Tr } \mathbf{S}).$$

Componenti successive. Per ottenere la k -esima componente si impone $\mathbf{u}_k^\top \mathbf{u}_j = 0$ per ogni $j < k$ e si massimizza nuovamente la varianza; gli autovettori di $\mathbf{S}$ formano quindi una base ortonormale $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_d]$. Troncando dopo i primi k vettori $\mathbf{U}_k \in \mathbb{R}^{d \times k}$ si ottiene la proiezione a bassa dimensionalità

$$\mathbf{Z} = \mathbf{U}_k^\top \mathbf{X} \in \mathbb{R}^{k \times n}, \quad \mathbf{X} \approx \mathbf{U}_k \mathbf{Z},$$

che preserva la percentuale di varianza

$$\sum_{j=1}^k \lambda_j / \sum_{j=1}^d \lambda_j.$$

Algoritmo sintetico.

1. **Centra** i dati: $\mathbf{X} \leftarrow \mathbf{X} - \mathbf{1} \boldsymbol{\mu}^\top$ con $\boldsymbol{\mu} = \frac{1}{n} \sum_i \mathbf{x}_i$.
2. **Covarianza**: $\mathbf{S} = \frac{1}{n} \mathbf{X} \mathbf{X}^\top$.
3. **Eigendecomposizione**: $\mathbf{S} \mathbf{U} = \mathbf{U} \text{diag}(\lambda_1, \dots, \lambda_d)$.
4. **Ordina** gli autovalori in ordine decrescente; seleziona k tramite soglia di varianza ($\gamma = 0.9-0.95$).
5. **Proietta**: $\mathbf{Z} = \mathbf{U}_k^\top \mathbf{X}$.

Osservazione 6.1. La formulazione appena vista coincide con la minimizzazione dell'errore quadratico di una decomposizione a rango k della matrice dei dati, anticipando il legame con la Singular Value Decomposition (vedi §6.3).

Esempio 6.1 (Dataset bidimensionale). In due dimensioni la prima componente corrisponde alla retta lungo la quale si proiettano i campioni minimizzando la somma delle distanze quadrate dai punti proiettati; il risultato è visivamente l'*asse di massima inerzia* del cloud di punti.

6.2 Autovalori e autovettori

Definizione 6.1 (Equazione agli autovalori). Un vettore non nullo $\mathbf{x} \in \mathbb{C}^d$ è *autovettore* di una matrice quadrata $\mathbf{A} \in \mathbb{C}^{d \times d}$ se esiste $\lambda \in \mathbb{C}$ tale che

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}.$$

Il numero λ è detto *autovalore*.

Osservazioni di base.

- La norma dell'autovettore è irrilevante: se $\mathbf{x}$ è soluzione, lo è anche $c\mathbf{x}$ per qualunque $c \neq 0$.
- Segno opposto: $\mathbf{x}$ e $-\mathbf{x}$ condividono lo stesso autovalore.
- Esempio funzionale: la derivata soddisfa $\frac{d}{dx}e^{ax} = ae^{ax}$; l'esponenziale è autovettore (autofunzione) dell'operatore derivata.

Trasformazioni di similarità. Per ogni matrice invertibile $\mathbf{T}$

$$\mathbf{B} = \mathbf{T}^{-1}\mathbf{A}\mathbf{T} \implies \mathbf{A} \text{ e } \mathbf{B} \text{ condividono gli autovalori.}$$

Casi notevoli.

- **Matrici ortogonali:** $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I} \implies$ gli autovalori hanno modulo uno, quindi $\lambda = \pm 1$.
- **Matrici diagonali o triangolari superiori:** gli autovalori coincidono con gli elementi sulla diagonale.
- **Matrici commutanti:** se $\mathbf{AB} = \mathbf{BA}$, allora $\mathbf{A}$ e $\mathbf{B}$ ammettono una stessa base di autovettori.

Matrici simmetriche. Se $\mathbf{A} = \mathbf{A}^\top$ allora:

- tutti gli autovalori sono *reali*;
- autovettori associati ad autovalori distinti sono ortogonali;
- esiste la decomposizione spettrale $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^\top$, con $\mathbf{X}$ ortogonale e $\mathbf{\Lambda}$ diagonale.

Quoziente di Rayleigh e teorema min-max. Per una matrice simmetrica vale

$$\lambda_{\min} \leq \frac{\mathbf{v}^\top \mathbf{A} \mathbf{v}}{\mathbf{v}^\top \mathbf{v}} \leq \lambda_{\max}, \quad \max_{\|\mathbf{v}\|_2=1} \mathbf{v}^\top \mathbf{A} \mathbf{v} = \lambda_{\max}.$$

Perché sono utili? Se gli autovettori formano una base, qualsiasi vettore $\mathbf{y}$ si espande come $\mathbf{y} = \sum_i \alpha_i \mathbf{x}_i$; moltiplicare per $\mathbf{A}$ equivale a scalare ciascuna direzione: $\mathbf{A}\mathbf{y} = \sum_i \alpha_i \lambda_i \mathbf{x}_i$. Questo fornisce una rappresentazione “diagonale” di operatori lineari complessi.

Calcolo numerico.

Power iteration convergenza al maggiore autovalore/eigenvector: $\mathbf{v}_{t+1} \leftarrow \mathbf{A}\mathbf{v}_t / \|\mathbf{A}\mathbf{v}_t\|$.

Inverse iteration applicata a $\mathbf{A}^{-1}$ (o tramite fattorizzazione LU) restituisce il minore autovalore.

Shift dato uno shift σ , si applica l'inverse iteration a $\mathbf{A} - \sigma\mathbf{I}$ per isolare autovalori vicini a σ .

Esempio 6.2 (Power iteration su matrice 2×2). Sia $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ e $\mathbf{v}_0 = (1, 0)^\top$. Dopo pochi passi la sequenza converge a $\mathbf{u} = \frac{1}{\sqrt{2}}(1, 1)^\top$, autovettore associato a $\lambda = 3$.

6.3 Singular Value Decomposition

Definizione. Per ogni matrice $\mathbf{A} \in \mathbb{R}^{m \times n}$ esiste la fattorizzazione

$$\boxed{\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top} \quad \begin{array}{ll} \mathbf{U} \in \mathbb{R}^{m \times m}, & \mathbf{U}^\top \mathbf{U} = \mathbf{I}_m \quad (\text{sin. sinistre}) \\ \mathbf{V} \in \mathbb{R}^{n \times n}, & \mathbf{V}^\top \mathbf{V} = \mathbf{I}_n \quad (\text{sin. destre}) \\ \mathbf{\Sigma} \in \mathbb{R}^{m \times n}, & \mathbf{\Sigma} = \text{diag}(\sigma_1, \dots, \sigma_\ell), \sigma_1 \geq \dots \geq \sigma_\ell \geq 0, \quad \ell = \min\{m, n\}. \end{array}$$

Proprietà chiave.

- Le *singular values* σ_i sono radici quadrate degli autovalori di $\mathbf{A}^\top \mathbf{A}$ (oppure $\mathbf{A} \mathbf{A}^\top$).
- Gli autovettori di $\mathbf{A}^\top \mathbf{A}$ formano le colonne di $\mathbf{V}$; quelli di $\mathbf{A} \mathbf{A}^\top$ formano le colonne di $\mathbf{U}$.
- Scritta come $\mathbf{A} \mathbf{V} = \mathbf{U} \mathbf{\Sigma}$, la SVD ricorda l'equazione agli autovalori, con la differenza che $\mathbf{\Sigma}$ è rettangolare.

Interpretazione geometrica. L'azione lineare di $\mathbf{A}$ può essere vista in tre step:

$$\mathbf{x} \xrightarrow{\mathbf{V}^\top} \text{rotazione/riflessione} \rightarrow \xrightarrow{\mathbf{\Sigma}} \text{scaling anisotropo} \rightarrow \xrightarrow{\mathbf{U}} \text{rotazione/riflessione}.$$

Il vettore $\mathbf{x}$ viene prima espresso nella base di destra ($\mathbf{V}$), poi dilatato lungo assi ortogonali di ampiezza σ_i e infine ricombinato nella base di sinistra ($\mathbf{U}$). L'unit sphere si trasforma così in un ellissoide.

Espansione in prodotti esterni.

$$\mathbf{A} = \sum_{i=1}^{\ell} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top, \quad \mathbf{u}_i, \mathbf{v}_i \text{ colonne corrispondenti di } \mathbf{U}, \mathbf{V}.$$

Approssimazioni a basso rango. Troncando dopo i primi $k < \ell$ termini

$$\mathbf{A} \approx \mathbf{A}_{(k)} := \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top,$$

si ottiene la miglior approssimazione di rango $\leq k$ in norma di Frobenius (teorema di Eckart-Young).

Pseudoinversa di Moore-Penrose. Se $\sigma_i > 0$ per $i \leq r$, $\mathbf{A}^\dagger = \mathbf{V} \mathbf{\Sigma}^\dagger \mathbf{U}^\top$, con $\mathbf{\Sigma}^\dagger = \text{diag}(\sigma_1^{-1}, \dots, \sigma_r^{-1})$.

Collegamento con la PCA. Data la matrice centrata dei dati $\mathbf{X} \in \mathbb{R}^{n \times d}$, la PCA coincide con la SVD ristretta:

$$\mathbf{X} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top, \quad \mathbf{S} = \frac{1}{n} \mathbf{X}^\top \mathbf{X} = \mathbf{V} \left(\frac{\mathbf{\Sigma}^2}{n} \right) \mathbf{V}^\top.$$

Gli autovettori $\mathbf{V}$ sono le componenti principali e σ_i^2/n le varianze corrispondenti.

Uso pratico.

- **Compressione:** riduzione delle dimensioni di immagini, documenti o segnali tramite il troncamento dei piccoli σ_i .
- **Denoising:** le componenti associate a σ_i piccoli catturano soprattutto rumore.
- **Risoluzione di sistemi sovra-/sotto-determinati** via pseudoinversa.

6.4 Principal Component Analysis

Obiettivo. Per un dataset di n punti $\mathbf{x}_i \in \mathbb{R}^d$ (raccolti nelle colonne di $\mathbf{X} \in \mathbb{R}^{d \times n}$ e **centrati** a media zero) vogliamo

trovare $k \leq d$ direzioni ortogonali $\mathbf{w}_1, \dots, \mathbf{w}_k$

che

- a) massimizzino la varianza dei dati proiettati;
- b) minimizzino l'errore di ricostruzione dopo la proiezione.

Formulazione varianza-massima. Per una direzione unitaria $\mathbf{w}$ la proiezione di tutti i punti è $\mathbf{X}^\top \mathbf{w}$; la varianza da massimizzare è

$$\|\mathbf{X}^\top \mathbf{w}\|_2^2 = \mathbf{w}^\top (\mathbf{X}\mathbf{X}^\top) \mathbf{w} = \mathbf{w}^\top \mathbf{C} \mathbf{w}, \quad \mathbf{C} \in \mathbb{R}^{d \times d} \text{ matrice di covarianza.}$$

Problema:

$$\max_{\|\mathbf{w}\|_2=1} \mathbf{w}^\top \mathbf{C} \mathbf{w}.$$

La soluzione è l'autovettore principale di $\mathbf{C}$; il valore ottimo coincide con l'autovalore corrispondente. Direzioni successive si ottengono imponendo ortogonalità (vincolo $\mathbf{w}_j^\top \mathbf{w} = 0$ per ogni $j < i$) e scegliendo gli autovettori restanti in ordine di autovalore decrescente.

Proiezione e ricostruzione. Indicando $\mathbf{W} = [\mathbf{w}_1 \dots \mathbf{w}_k] \in \mathbb{R}^{d \times k}$, $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_k$, si ha

$$\boxed{\mathbf{Z}^\top = \mathbf{X}^\top \mathbf{W} \quad (\text{proiezione}), \quad \mathbf{X} \approx \mathbf{W} \mathbf{Z} \quad (\text{ricostruzione}).}$$

Interpretazione geometrica. La matrice $\mathbf{W}$ definisce un sottospazio di dimensione k che “cattura” la massima energia (varianza) dei dati; l'errore è misurato *ortogonalmente* a tale sottospazio — a differenza della regressione lineare, dove l'errore è valutato lungo un asse prefissato.

Algoritmo (sintesi).

1. Centra le osservazioni: $\mathbf{X} \leftarrow \mathbf{X} - \underline{\mathbf{1}} \boldsymbol{\mu}^\top$.
2. Calcola $\mathbf{C} = \mathbf{X}\mathbf{X}^\top$.
3. Ottieni gli autovettori di $\mathbf{C}$ e scegli i primi k .
4. Proiezione: $\mathbf{Z}^\top = \mathbf{X}^\top \mathbf{W}$.
5. (Facoltativo) ricostruzione: $\tilde{\mathbf{X}} = \mathbf{W} \mathbf{Z}$.

Modello generativo. Una volta stimata $\mathbf{W}$, nuovi campioni a bassa dimensione $\mathbf{z}_{\text{new}} \in \mathbb{R}^k$ generano dati nello spazio originale tramite $\mathbf{x}_{\text{new}} = \mathbf{W} \mathbf{z}_{\text{new}}$.

Riassunto. La PCA è dunque un metodo di compressione *lossy* che sostituisce i dati con una loro proiezione in un sottospazio “più informativo possibile” in senso di varianza; si risolve tramite l'analisi spettrale della matrice di covarianza e fornisce una rappresentazione adatta sia alla riduzione di dimensionalità sia alla generazione di nuovi dati in maniera controllata.

7 Dimensionality reduction

7.1 Spazi metrici

Dati due punti x, y la loro distanza è data dalla scelta della metrica. Una metrica è una funzione $d_{\mathcal{M}} : X \times X \rightarrow \mathbb{R}^+$ che soddisfa le seguenti proprietà:

- $d_{\mathcal{M}}(x, y) = 0 \iff x = y$
- $d_{\mathcal{M}}(x, y) = d_{\mathcal{M}}(y, x)$
- $d_{\mathcal{M}}(x, y) \leq d_{\mathcal{M}}(x, z) + d_{\mathcal{M}}(z, y)$

Alcuni esempi di metriche:

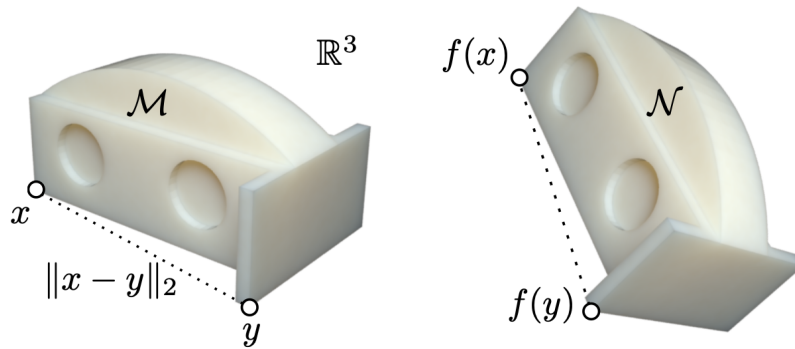
- $\mathcal{X} = \mathbb{R}, d_{\mathcal{X}}(x, y) = |x - y|$
- $\mathcal{X} = \mathbb{A} \subset \mathbb{R}^k, d_{\mathcal{X}}(x, y) = \|x - y\|_2$
- $\mathcal{X} = \mathbb{R}^k, d_{\mathcal{X}}(x, y) = \log(|x - y| + 1)$
- $\mathcal{X}$ qualunque insieme, $d_{\mathcal{X}}(x, y) = \begin{cases} 0 & \text{se } x = y \\ 1 & \text{altrimenti} \end{cases}$ (metrica discreta)
- $\mathcal{X} = \mathcal{A} \times \mathcal{B}, d_{\mathcal{X}}((a_1, b_1), (a_2, b_2)) = \sqrt{d_{\mathcal{A}}(a_1, a_2)^2 + d_{\mathcal{B}}(b_1, b_2)^2}$ (in uno spazio prodotto)

Se $\mathcal{A}$ è uno spazio metrico e $\mathcal{X} \subset \mathcal{A}$, allora $\mathcal{A}$ si chiama **spazio ambiente** per $\mathcal{X}$.

Definizione 7.1 (Isometria). Siano $(\mathcal{M}, d_{\mathcal{M}})$ e $(\mathcal{N}, d_{\mathcal{N}})$ due spazi metrici. Un'isometria è una mappa biettiva che soddisfa la proprietà:

$$d_{\mathcal{M}}(x, y) = d_{\mathcal{N}}(f(x), f(y)) \quad \forall x, y \in \mathcal{M}$$

Le isometrie sono dunque tutte le mappe che preservano le distanze.



Possiamo quantificare in modo continuo anche quanto siamo “lontani” da una **isometria rigida**. Si parla dunque di **quasi isometria**.

$$d_{\mathcal{M}}(x, y) \approx d_{\mathcal{N}}(f(x), f(y))$$

Osservazione 7.1. L'isometria è una relazione di equivalenza.

Per cui spazi metrici isometrici sono lo stesso spazio.

Definizione 7.2 (Embedding isometrici). Un embedding isometrico è una mappa $f : \mathcal{X} \rightarrow \mathcal{Z}$ che preserva la metrica per ogni $x, y \in \mathcal{X}$:

$$d_{\mathcal{Z}}(f(x), f(y)) = d_{\mathcal{X}}(x, y)$$

Quando cerchiamo embedding isometrici arriviamo a una possibile soluzione, che rappresenta una classe di equivalenza di mappe isometriche.

Cercheremo dunque di definire un algoritmo che ci permetta di trovare almeno un “quasi embedding” che ci permetta di rappresentare i dati in uno spazio metrico più basso-dimensionale. La distorsione indotta da f può essere quantificata in diversi modi:

- Relativa:

$$\frac{d_{\mathcal{Y}}(f(x_1), f(x_2))}{d_{\mathcal{X}}(x_1, x_2)} \approx 1 \quad \forall x_1, x_2 \in \mathcal{X}$$

- Assoluta:

$$|d_{\mathcal{Y}}(f(x_1), f(x_2)) - d_{\mathcal{X}}(x_1, x_2)| \approx 0 \quad \forall x_1, x_2 \in \mathcal{X}$$

Ma possiamo anche costruirne una a seconda del problema. Se $d_{\mathcal{Y}} = \|\cdot\|_2$ chiamiamo la forma embeddata (l'immagine di f) forma **canonica** di $\mathcal{X}$.

7.2 Multidimensional scaling

Consideriamo lo stress quadratico:

$$f = \arg \min_{f: \mathcal{X} \rightarrow \mathbb{R}^k} \sum_{i>j} |d_{\mathcal{X}}(x_i, x_j) - \|f(x_i) - f(x_j)\|_2|^2$$

definiamo $\mathbf{z}_i = f(x_i)$ e scriviamo questi vettori in una matrice $\mathbf{Z} \in \mathbb{R}^{n \times k}$ (n punti del dataset). Otteniamo il problema equivalente:

$$\mathbf{Z} = \arg \min_{\mathbf{Z} \in \mathbb{R}^{n \times k}} \sum_{i>j} |d_{\mathcal{X}}(x_i, x_j) - d_{ij}(\mathbf{Z})|^2$$

dove $d_{ij}(\mathbf{Z}) = \|\mathbf{z}_i - \mathbf{z}_j\|_2$.

7.3 Stochastic Neighbor Embedding

Ha lo scopo di visualizzare dati. Consideriamo similarità locali tra punti usando una distribuzione di probabilità.

$$p_{ij} = e^{-\frac{\|x_i - x_j\|^2}{2\sigma^2}} \in (0, 1), \quad p_{ij} = \frac{p_{ij}}{\sum_{k \neq i} p_{ik}}$$

si dice che è una misura locale, visto che valori che possono avere distanze elevate tra loro, ma che hanno distanza simile dal punto che considero, avranno probabilità simili da quello che considero. Per cui invece di distanze euclidee

$$\min_{z_i \in \mathbb{R}^k} \sum_{i,j} (\|x_i - x_j\|^2 - \|z_i - z_j\|^2)^2$$

possiamo usare la divergenza di Kullback-Leibler:

$$\min_{z_i \in \mathbb{R}^k} \sum_{i,j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

con $q_{ij} = e^{-\|z_i - z_j\|^2}$, $q_{ij} = \frac{q_{ij}}{\sum_{k \neq i} q_{ik}}$.

La divergenza di Kullback-Leibler è una relazione non simmetrica, per cui se p_{ij} è grande e q_{ij} è piccolo, allora la penalty è grande. Se invece p_{ij} è piccolo e q_{ij} è grande, allora la penalty è piccola.

Possiamo scegliere per ogni punto un valore di σ_i (perplexità) a seconda di quanti vicini vogliamo per ogni punto.

7.4 t-SNE

Abbiamo visto che punti lontani nella gaussiana hanno penalty nulle. Usando una distribuzione di Cauchy nello spazio target abbiamo distribuzioni che hanno code più lunghe, per cui possiamo rappresentare meglio i punti lontani.

$$q_{ij} = (1 + \|z_i - z_j\|^2)^{-1} \quad q_{ij} = \frac{q_{ij}}{\sum_{k \neq i} q_{ik}}$$

Si vuole dunque minimizzare:

$$D_{KL}(P \parallel Q) = \sum_{i,j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Per SNE (Gaussiane p e q) si ha:

$$\frac{\partial D_{KL}}{\partial z_i} = 2 \sum_j (p_{ij} - q_{ij} + p_{ji} - q_{ji})(z_i - z_j)$$

Per t-SNE (Gaussiana p e Cauchy q) si ha:

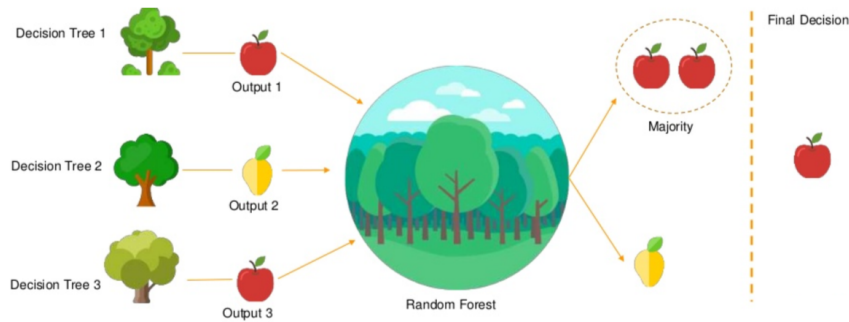
$$\frac{\partial D_{KL}}{\partial z_i} = 4 \sum_j (p_{ij} - q_{ij})(z_i - z_j)(1 + \|z_i - z_j\|^2)^{-1}$$

8 Metodi Ensemble

L'idea è di combinare più modelli **weak learners** per ottenere un modello **strong ensemble**. L'obiettivo è di aumentare le performance e la robustezza.

Definizione 8.1 (Weak Learners). Modelli che performano meglio della previsione randomica. Sono semplici e veloci, ma hanno un **bias alto** (underfittano) e varianza bassa.

Possiamo dunque ridurre il bias aggregando più modelli.



Costruiamo dunque alberi di decisione multipli e ogni albero da un output. Le predizioni sono dunque combinate in una predizione finale.

Definizione 8.2 (Decision Tree). Un albero di decisione è un albero binario in cui ogni ramo rappresenta una decisione possibile. Un cammino attraverso l'albero rappresenta una sequenza di azioni. I dati sono dunque partizionati in sottoinsiemi dalla radice verso le foglie.

Possiamo considerare un semplice modello che ha un livello di decisione, per cui splitta i dati basandosi su una singola feature. Lo chiamiamo **Decision Stump**.

L'operazione da fare è dunque quella di guadagnare la maggior informazione possibile per ogni nodo di decisione. La foglia corrisponderà dunque alla decisione finale.

Usando la misura di entropia possiamo conoscere la quantità di informazione data una distribuzione di dati. Lo split dovrebbe essere più **discriminativo** possibile: massimizzare il guadagno $E_2 - E_1$.

Il problema da risolvere è dunque quello di classificare in base a una o più features. Si parte da un set di dati e ne si definiscono le features su cui basare le nostre decisioni.

A ogni nodo scegliamo la domanda che massimizza il guadagno di informazione e possiamo continuare fino a che il dataset è vuoto o si è raggiunto un'accuratezza sufficiente. La foglia corrisponderà a una condizione in cui lo split non è più possibile (zero entropia) o richiesto. A ogni foglia l'accuratezza rispetto alla label l si misura come:

$$\text{accuracy} = \frac{\# \text{ dati con label } l}{\# \text{ dati}}$$

Osservazione 8.1. *Nodi profondi rappresentano interazioni più complesse tra le features.*

- A profondità 2 lo split è fatto su una feature
- A profondità 3 lo split è fatto su due features
- A profondità k lo split è fatto su $k - 1$ features

Alberi troppo profondi possono catturare interazioni complesse, ma sono soggetti a **overfitting**.

Osservazione 8.2. *Per problemi di **classificazione***

- Ogni albero dà in output una distribuzione su tutte le classi
- Gli “split interni” massimizzano il guadagno di informazione
- Le foglie contengono la classe maggioritaria

*Per problemi di **regressione***

- Ogni albero dà in output un valore continuo
- Gli “split interni” minimizzano l'errore quadratico medio (**MSE**)
- Le foglie contengono il valore medio dei punti

Ad esempio, dato un training data di cellulari e delle features (e.g. luminosità, numero di app in background, batteria) e un valore da predire (e.g. temperatura). Dato che la temperatura è un valore continuo, usiamo una **regression forest**.

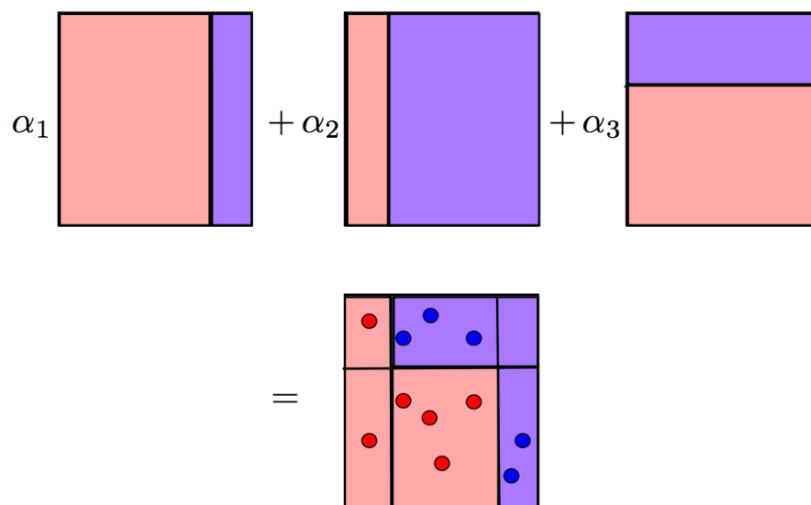
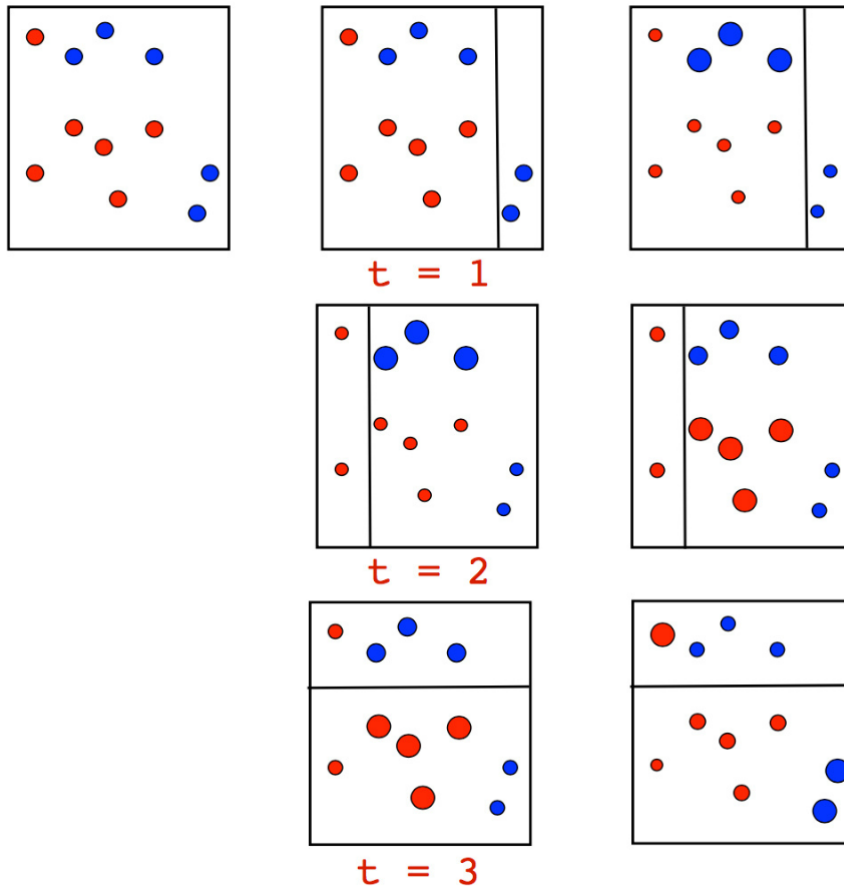
- Crea uno split, con due nodi figli
- In ogni nodo figlio, misuriamo la temperatura media
- Per ogni cellulare nel nodo figlio, calcoliamo la differenza tra la temperatura osservata e la temperatura media e consideriamo la media delle differenze
- Computiamo la media pesata $\frac{n_{\text{left}}}{n} \times \text{MSE}_{\text{left}} + \frac{n_{\text{right}}}{n} \times \text{MSE}_{\text{right}}$
- Manteniamo lo split che ha l'MSE più basso

1. Splittiamo i dati in partizioni random (**bagging**)
2. Trainiamo un albero per ogni partizione. Ogni albero ha domande differenti, altezze diverse e foglie con accuratze differenti, ma vogliono risolvere lo stesso problema.
3. dato un nuovo punto, lo instradiamo attraverso ogni albero e ogni albero dà la sua predizione.
4. Le predizioni sono combinate in una predizione finale (**forest prediction**)

Boosting

Vogliono ridurre il bias basandosi sugli errori dei modelli precedenti. L'idea è quella di trainare modelli **weak learners**, concentrandosi ogni volta sui punti che sono stati classificati male e quindi combinandoli.

- **AdaBoost**: ad ogni iterazione, pesiamo nuovamente i dati del training set per concentrarsi sui punti classificati male



e gli α_i sono proporzionali all'accuratezza di ogni **weak learner**. Si può vedere come AdaBoost tende a overfittare.

- **Gradient Boosting**: illustriamo con una riduzione ai minimi quadrati per $F : X \rightarrow \mathbb{R}$.

$$L(y, F(x)) = \sum_{i=1}^n (y_i - F(x_i))^2$$

E iteriamo i weak learners iterando per $m = 0, 1, \dots$:

- Trainiamo un weak learner F_m . Per $m = 0$ definiamo $F_m(x_i) = \frac{1}{n} \sum_{k=1}^n y_k$
- Consideriamo $F_{m+1}(x_i) = F_m(x_i) + \underbrace{h_{m+1}(x_i)}_{\text{residuo}} = y_i$
- Ora il nuovo modello h_{m+1} è addestrato per fare in modo che $h_{m+1}(x_i) \approx y_i - F_m(x_i)$

Ogni F_{m+1} tenta di correggere gli errori dei suoi predecessori F_m . La soluzione finale è data dalla combinazione:

$$F_0(x) + \sum_m h_m(x)$$

Il motivo per cui si chiama **gradient boosting** è che data una **MSE loss**

$$L(y, F(x)) = \sum_i (F(x_i) - y_i)^2$$

la sua derivata (negativa) rispetto alla predizione $F(x)$ è:

$$-\frac{\partial L(y, F(x))}{\partial F(x)} = 2 \sum_i (y_i - F(x_i)) = 2 \sum_i h_{m+1}(x_i)$$

per questa ragione le derivate $-\frac{\partial L(y, F(x))}{\partial F(x)}$ sono chiamati **pseudo-residui**.

Effettivamente $h_{m+1} \propto -\frac{\partial L}{\partial F}$, e ricorda l'approccio usato dal gradient descent.

$$F_{m+1}(x) = F_m(x) + \gamma h_{m+1}(x)$$

dove γ è un parametro di learning rate. In altre parole

- Gradient boosting è inizializzato con un modello F_0 (che sarà pessimo)
- Invece di computare $\frac{\partial L}{\partial F}$, fittiamo un modello weak learner h_{m+1} a F_0
- Invece di fissare γ , lo computiamo con una ricerca lineare:

$$\gamma_{m+1} = \arg \min_{\gamma} L(y, F_m(x) + \gamma h_{m+1}(x))$$

Osservazione 8.3. Il **weak learner** $h_{m+1}(x)$ è addestrato usando gli **pseudo-residui** per i training set $\{(x_i, r_i)\}$ e **non** usando i dati originali $\{(x_i, y_i)\}$.

Gradient Boosting nella pratica

- **Gradient Boosting nella pratica**:

- Progettato per apprendere modelli a valore scalare F .
- Per la classificazione multinomiale, tipicamente si utilizza il paradigma **one-vs-rest**.
- Si calcolano gli **pseudo-residui** come le derivate:

$$r_i = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$$

e poi si adattano i **weak learners** a questi.

- Concettualmente ci sono due loss:
 - * La loss L definita sui dati originali $\{(x_i, y_i)\}$, che viene minimizzata su tutte le iterazioni.
 - * Una loss interna per adattare i residui $\{(x_i, r_i)\}$ tramite i weak learners. Questa loss potrebbe non essere definita esplicitamente (e.g. foreste di regressione).
- Questo assicura che ogni modello successivo nel processo di boosting si concentri sulla correzione degli errori dei modelli precedenti.

Osservazione 8.4 (Gradient Boosting vs Deep Learning). *Vediamo le differenze tra gradient boosting e deep learning:*

- Gradient Boosting**
- Funziona bene con dati eterogenei tabulari
 - Richiede meno dati di training
 - Più facile da interpretare
 - Più veloce da addestrare

- Deep Learning**
- Funziona molto bene su grandi dataset
 - Assorbe le features in maniera automatica (e.g. patterns spaziali)
 - Permette il **transfer learning** con il **fine-tuning**
 - Più flessibile per più task